

Deep Learning

- Linear/Logistic regression gradient descent

Layers:

- Linear
- ReLU, Sigmoid
- Softmax
- X-ent, X-ent w/ logits
- Square loss
- add, mul, sum, max
- Conv, max-pool
 - padding, stride, dilation
- batch norm & residual connections

CNN:

- general architecture
- gradients problem & solutions
- applications

RNN:

- simple RNN, $\frac{\partial L}{\partial b}$, $\frac{\partial L}{\partial w}$
- LSTM & thresholding
- Applications

Pretraining

Optimisation:

- SGD, why zig-zag, momentum & Nesterov
- Newton, problem, Gauss Newton
- Curvature Matrix approximations

Attention & Memory:

- Implicit; Jacobian
- Explicit; glimpse, soft vs hard

DL for NLP:

- word embedding
- LBL/NLM/k-gram/RNN-LM
- seq-to-seq

DL

Linear Regression:

$$L(\underline{w}, S) = \frac{1}{2} \sum (y_i - \underline{w}^T \underline{x}_i)^2$$

$$\nabla_{\underline{w}} L = \sum (y_i - \underline{w}^T \underline{x}_i) \underline{x}_i$$

Logistic Regression:

$$L(\underline{w}, S) = -\sum (y_i \log \sigma(\underline{w}^T \underline{x}_i) + (1 - y_i) \log (1 - \sigma(\underline{w}^T \underline{x}_i)))$$

$$\begin{aligned} \nabla_{\underline{w}} L &= -\sum (y_i (1 - \sigma(\underline{w}^T \underline{x}_i)) - (1 - y_i) \sigma(\underline{w}^T \underline{x}_i)) \underline{x}_i \\ &= \sum (\sigma(\underline{w}^T \underline{x}_i) - y_i) \underline{x}_i \end{aligned}$$

Layer	y	$\frac{\partial L}{\partial \underline{x}}$	$\frac{\partial L}{\partial \underline{\theta}}$
Linear	$\underline{w}^T \underline{x} + b$	$\frac{\partial L}{\partial \underline{y}} \underline{w}$	$\frac{\partial L}{\partial \underline{w}} = \left(\frac{\partial L}{\partial \underline{y}}\right)^T \underline{x}^T$ $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \underline{y}}$
ReLU	$y_i = \max(x_i, 0)$	$\frac{\partial L}{\partial x_i} = (y_i > 0)$	
Sigmoid			
Softmax	$y_i = \frac{e^{x_i}}{\sum e^{x_j}}$	$\underline{s} - \sum s_j (s_j = \frac{\partial L}{\partial y_j} y_j)$	
Cross Ent	$-\sum p_i \log x_i$	$\frac{\partial L}{\partial x_i} = -\frac{p_i}{x_i}$	
(w/ logits)	$-\sum p_i \log \left(\frac{e^{x_i}}{\sum e^{x_j}}\right)$	$\frac{\partial L}{\partial x_i} = y - p$	
Square loss	$\ \underline{t} - \underline{x}\ ^2$	$-2\ \underline{t} - \underline{x}\ $	
Add	$\underline{a} + \underline{b}$	$\frac{\partial L}{\partial \underline{a}} = \frac{\partial L}{\partial \underline{y}}$	
Mul	$\underline{a} \odot \underline{b}$	$\frac{\partial L}{\partial \underline{a}} = \underline{b} \odot \frac{\partial L}{\partial \underline{y}}$	
Sum	$\sum x_i$	$\frac{\partial L}{\partial \underline{x}} = \frac{\partial L}{\partial \underline{y}} \cdot \underline{1}^T$	
Max	$\max x_i$	$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial \underline{y}}$ if $x_i = \max(\underline{x})$ else 0	

CNN:

input $\downarrow$ $\uparrow$ dilation

$$\text{output} \rightarrow M = \left\lceil \frac{N + 2p - k'}{s} \right\rceil + 1, \quad k' = k + (k-1)(r-1)$$

s = stride
p = padding

• SAME padding: $p = (k-1)/2$

$[(\text{conv} \times m) \rightarrow \text{pool}] \times n \rightarrow \text{linear} \times k \rightarrow \text{softmax}$

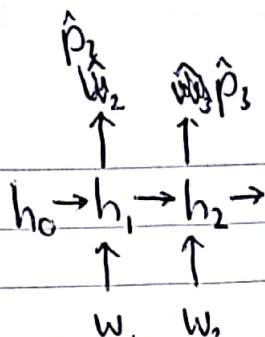
• main workhorse: 3x3 conv

• Vanishing/exploding gradients:

- add batch norm & residual connections

• careful weight initialization

• RNN:

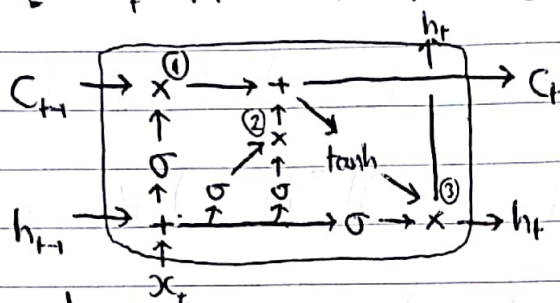


$$h_t = \tanh(W h_{t-1} + U w_t)$$

$$\hat{p}_{t+1} = \text{softmax}(B h_t)$$

$$L(W, U, B) = -\sum_t \log p(w_t | w_{1:t-1})$$

$$\nabla_B L = \sum_t (p_t - \text{label}_t) h_t^T, \quad \nabla_W L = \sum_t \sum_{k=1}^K \frac{\partial L_t}{\partial p_k} \frac{\partial p_k}{\partial h_t} \frac{\partial h_t}{\partial W} \frac{\partial h_k}{\partial W}$$



• helps long term dependencies
& vanishing/exploding
gradients
(can also threshold gradients)

• Optimisation:

• GD: $\Theta_{k+1} = \Theta_k - \alpha_k \nabla_{\Theta} l |_{\Theta_k}$

• implicitly approximates $\underline{H}$ by $\underline{L} \underline{I}$; overestimate curvature; zig-zags

• Momentum: $\Theta_{k+1} = \Theta_k + \eta V_{k+1}, \quad V_{k+1} = \eta V_k - \alpha_k \nabla_{\Theta} l |_{\Theta_k}$

• fix zig-zagging; Nesterov better theoretical bound, similar in practice

• 2nd order:

• Newton: $\Theta_{k+1} = \Theta_k - \alpha \underline{H}^{-1} \nabla_{\Theta} l |_{\Theta_k}$

• $\underline{H}$ under-estimates curvature outside small radius

• can limit to r ; replace $\underline{H}$ with $(\underline{H} + \lambda \underline{I})$

• or use other matrix, eg $\underline{G} = \sum \underline{J}_i^T \underline{H}_i \underline{J}_i$, $\underline{J}_i = \text{Jacobian of } f(x, \Theta) \text{ wrt } \Theta$
• $\underline{G}$ works better in practice (always PSD)

$\underline{H}_i = \text{Hessian of } l(y, \hat{y}) \text{ wrt } \hat{y}$

• Curvature matrix approximations:

• $\underline{H}: D \times D$, $\underline{H}^{-1}: O(D^3)$ (& $O(D^2)$ storage)

• too expensive; use approximations to speed up

• diagonal & empirical Fisher matrix (RMSProp, Adam)

• low rank approx (L-BFGS)

• No asymptotic improvement from stochastic 2nd order, but improvement before asymptote $\Rightarrow$ can still be good to imitate

• Attention & Memory (RNN):

• Implicitly given by Jacobian; $\underline{J}_{ij} = \frac{\partial y_i}{\partial x_j}$

• Explicitly done by attention model using network output + extra data to make a glimpse; then fed back into network as input

• Hard glimpse not differentiable; train by RL. Soft glimpse is; GD works good!

COMP6113 Advanced Topics in Machine Learning

Deep Learning

Supervised Learning

$$x \rightarrow [f] \rightarrow y \quad \begin{cases} y \in \{-1, +1\}, \text{classification} \\ y \in \mathbb{R}, \text{regression} \end{cases}$$

• Generalisation:

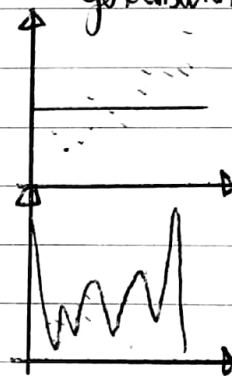
- performance on unseen data
 - empirical risk minimization: minimize training error
 - validation: hold out data to get unbiased ~~error~~ estimator
- biased! $\downarrow$
eg cross validation $\downarrow$
- $$\underbrace{\text{validation error}}_{\text{what we care about}} = \underbrace{\text{training error}}_{\text{error}} + \underbrace{\left(\frac{\text{validation error} - \text{training error}}{\text{error}} \right)}_{\text{generalisation error}}$$

• Underfitting:

- error driven by approximation
- high bias, low variance

• Overfitting:

- error driven by generalisation
- low bias, high variance



• Early stopping:

- control degree of overfitting in gradient iterative optimisation methods

• Regularisation:

- impose a ~~stable~~ preference for simpler hypothesis
- weight decay, (L1 or L2), dropout...

• Gradient descent:

$$\underline{w} \leftarrow \underline{w} - \eta \nabla L(\underline{w})$$

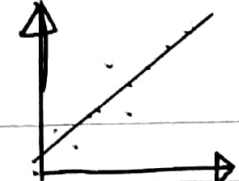
- useful when no closed form solution / dataset too large / high dimensional to practically solve
- often practical to use a mini-batch S_j to approximate $\nabla L(\underline{w})$, where $L(\underline{w}) = \sum_i f_i(\underline{w})$

loss function to minimise

$$\nabla L(\underline{w}) \approx \sum_{i \in S_j} \nabla f_i(\underline{w})$$

- minibatch size trades off computational cost & variance

• Linear Regression:



$\hat{y}(\underline{w}, \underline{x}) = \underline{w}^T \underline{x}$, $\ell(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$

$L(\underline{w}, S) = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}(\underline{w}, \underline{x}_i))^2$

Squared error = Gaussian prior noise

$\nabla \ell = y - \hat{y}$

$\nabla_{\underline{w}} L(\underline{w}, S) = \sum_i (y_i - \underline{w}^T \underline{x}_i) \underline{x}_i$

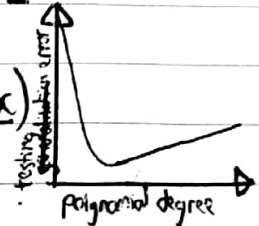
regularisation

→ • weight decay: $L'(\underline{w}, S) = L(\underline{w}, S) + \lambda \|\underline{w}\|^2$ ← Gaussian prior on weights

• Non-linear basis functions:

• $\Phi(\underline{x}) = (1, x, x^2)$, $\hat{y}(\underline{w}, \underline{x}) = \underline{w}^T \Phi(\underline{x})$

• provides richer hypothesis space



• Logistic Regression: $y \in \{0, 1\}$

$z(\underline{w}, \underline{x}) = \underline{w}^T \underline{x}$, $\hat{p}(z) = \frac{1}{1 + e^{-z}}$

cross
entropy
loss

→ $\ell(y, \hat{p}) = y \log \hat{p} + (1 - y) \log (1 - \hat{p})$

$L_{\underline{w}}(\underline{w}, S) = - \sum_i \ell(y_i, \hat{p})$

$= \sum_i \log(1 + e^{\underline{w}^T \underline{x}_i}) - y_i \underline{w}^T \underline{x}_i$

$\nabla_{\underline{w}} L(\underline{w}, S) = \sum_i \frac{x_i e^{\underline{w}^T \underline{x}_i}}{1 + e^{\underline{w}^T \underline{x}_i}} - y_i \underline{x}_i$

$= \sum_i \left(\frac{1}{1 + e^{-\underline{w}^T \underline{x}_i}} - y_i \right) \underline{x}_i$

Neural Networks: Foundations

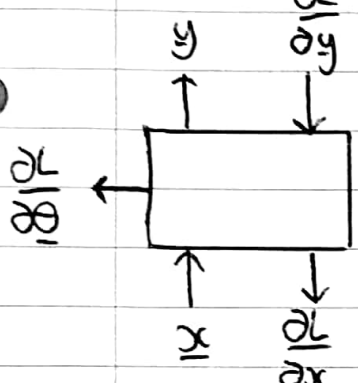
- Composition of linear transforms + non-linear functions
 - Learn by chain rule derivatives & SGD on loss function
 - each node in compute graph has:
 - forward-pass: $\underline{x} \rightarrow y, \frac{\partial L}{\partial \underline{x}} \rightarrow \frac{\partial L}{\partial y}$ (computing output & loss)
 - backward-pass: $\frac{\partial L}{\partial y} \rightarrow \frac{\partial L}{\partial \underline{x}}$ (backpropagation)
 - compute gradients: $\underline{x}, \frac{\partial L}{\partial y} \rightarrow \frac{\partial L}{\partial \theta}$ (for param updates)

Module Zoo:

Linear layer: (affine linear transform)

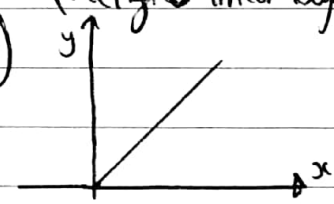
$$y = \underline{W} \underline{x} + b$$

$$\frac{\partial L}{\partial \underline{x}} = \frac{\partial L}{\partial y} \underline{W}$$

$$\frac{\partial L}{\partial \underline{W}} = \left(\frac{\partial L}{\partial y} \right)^T \underline{x}^T, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial y}$$


ReLU: (rectified linear layer; non-linearity)

$$y_i = \max(x_i, 0)$$

$$\frac{\partial L}{\partial x_i} = (y_i > 0)$$


Softmax:

$$y_i = \frac{e^{x_i}}{\sum e^{x_j}}$$

$$\frac{\partial L}{\partial x} = \underline{S} - \sum_i s_i, \quad s_i = \frac{\partial L}{\partial y_i} y_i$$

Cross-entropy loss:

p = label probabilities

$$y = - \sum_i p_i \log x_i$$

$$\frac{\partial L}{\partial x_i} = \frac{-p_i}{x_i}$$

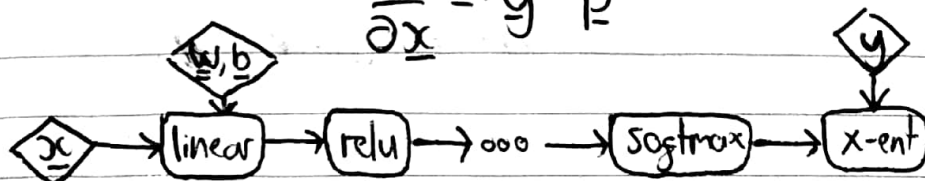
good to prevent overflow

Cross-entropy loss w/ logits:

$$y = - \sum_i p_i \log \left(\frac{e^{x_i}}{\sum_j e^{x_j}} \right)$$

$$\frac{\partial L}{\partial x} = y - p$$

(softmax + Xent loss in 1)



element
wise

• Add: $y = \underline{a} + \underline{b}$ $\frac{\partial L}{\partial \underline{a}} = \frac{\partial L}{\partial \underline{b}}$

• Mul: $y = \underline{a} \odot \underline{b}$ $\frac{\partial L}{\partial \underline{a}} = \underline{b} \odot \frac{\partial L}{\partial \underline{b}}$

group
wise

• Sum: $y = \sum x_i$ $\frac{\partial L}{\partial \underline{x}} = \frac{\partial L}{\partial y} \mathbf{1}^T$

• Max: $y = \max\{x_i\}$ $\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y}$ if x_i max, else 0

• Leaky-ReLU:

• add problem of 0-gradient on ReLU for $x_i < 0$ (sells asleep)

• Squared loss:

$y = \|\underline{t} - \underline{x}\|^2$ $\frac{\partial L}{\partial \underline{x}} = -2(\underline{t} - \underline{x})^T$

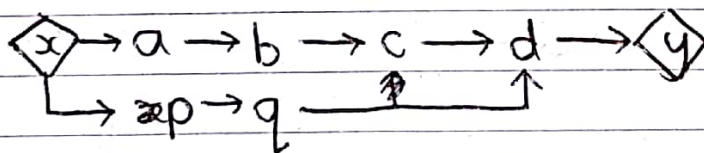
• Optimisation:

• Forward pass to find loss

• Backward pass to find gradients w.r.t. inputs

• Compute gradients w.r.t. parameters & update by gradient descent

(backtrack thru
forward execution pass)



$$y = d(c(b(a(x)), q(p(x))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial a}{\partial x} \cdot \frac{\partial b}{\partial a} \cdot \frac{\partial c}{\partial b} \cdot \frac{\partial d}{\partial c} \cdot \frac{\partial y}{\partial d} + \frac{\partial p}{\partial x} \cdot \frac{\partial q}{\partial p} \cdot \frac{\partial d}{\partial q} \cdot \frac{\partial y}{\partial d}$$

←

←
backprop

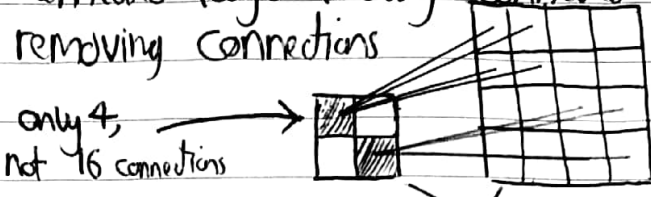
• Issues:

• Overfitting: regularisation (weight decay, dropout)

• Initialisation: scaling, batch-norm

• Hyperparameters: random vs grid search

Image Recognition with Convolutional Networks

- Aim for translational invariance
 - make fully connected layer locally connected by sharing weights & removing connections
- 
- only 4, not 16 connections
- Share weights
- not 'flat' in practice; have multiple channels (eg RGB=3 channels)

• Convolutional layer:

$$y = \sum_{i \in \text{Receptive field}} w_i x_i + b = \underline{w} * \underline{x} + \underline{b}$$

('Slide' receptive field over image & compute dot product)

- for $N \times N$ input & $k \times k$ kernel, output is $M \times M$, where $M = N - k + 1$

• Padding:

- VALID: no padding

- SAME: pad with $p = (k-1)/2$ zeros on all sides so output same size as input

• Stride:

- filter takes steps bigger than 1

- reduce computational cost / spatial resolution, & invariance to local translation

• Dilation:

- rate r = enlarge receptive field but keep same num parameters

$$k' = k + (k-1)(r-1)$$

$$M = \left\lceil \frac{N + 2p - k'}{s} \right\rceil + 1, \text{ for input } N \times N, p \text{ padding, } k' \text{ dilated kernel size \& } s \text{ stride length}$$

• Deconvolution:

- swap direction of convolution; increases resolution

- used in backprop

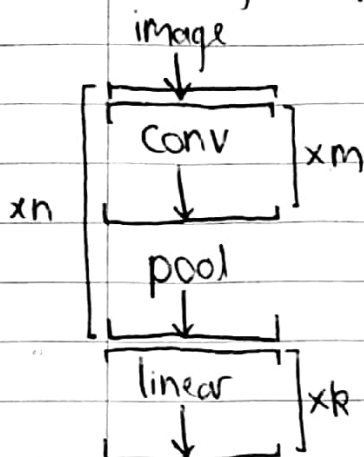
• Pooling layer:

- reduces spatial resolution

- average, or max pooling over receptive field

- can do global pooling too; full invariance to location

• Building deep networks:



- usually follow layers with a ReLU
- deeper usually better; more invariances & non-linearities
- typically build up no. channels over depth, before linear, FC layers

• main workhorse: 3×3 convolution layers

• stacking these up gives effective larger receptive fields

• Training:

• hard to train very deep nets; vanishing or exploding gradients

• Fixes:

• Weight initialisation:

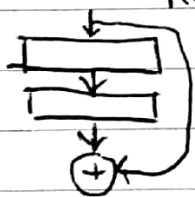
• 0 mean Gaussian, low variance, or, eg, variance to preserve gradient magnitude

• Batch normalisation:

• layer in network

• normalise input to zero mean, unit variance

• Residual connections:



• very deep networks get higher training error, as hard to learn identity function

• add connections which skip layers

• also improves gradient flow, as can skip layers

Sequences & Recurrent Neural Nets

$$\begin{array}{c}
 \hat{y}_2 \quad \hat{y}_3 \\
 \uparrow \quad \uparrow \\
 h_0 \rightarrow h_1 \rightarrow h_2 \rightarrow \dots \\
 \uparrow \quad \uparrow \\
 w_{x_1} \quad w_{x_2}
 \end{array}
 \quad
 \begin{array}{l}
 h_t = \tanh(W h_{t-1} + U x_t) \\
 p(\underline{w}_{t+1}) = \text{softmax}(\underline{B} h_t) \\
 L(\underline{W}, \underline{U}, \underline{B}) = \sum_{t=1}^T L_t
 \end{array}$$

$$L_t = -\log p(w_t | w_{1:t-1})$$

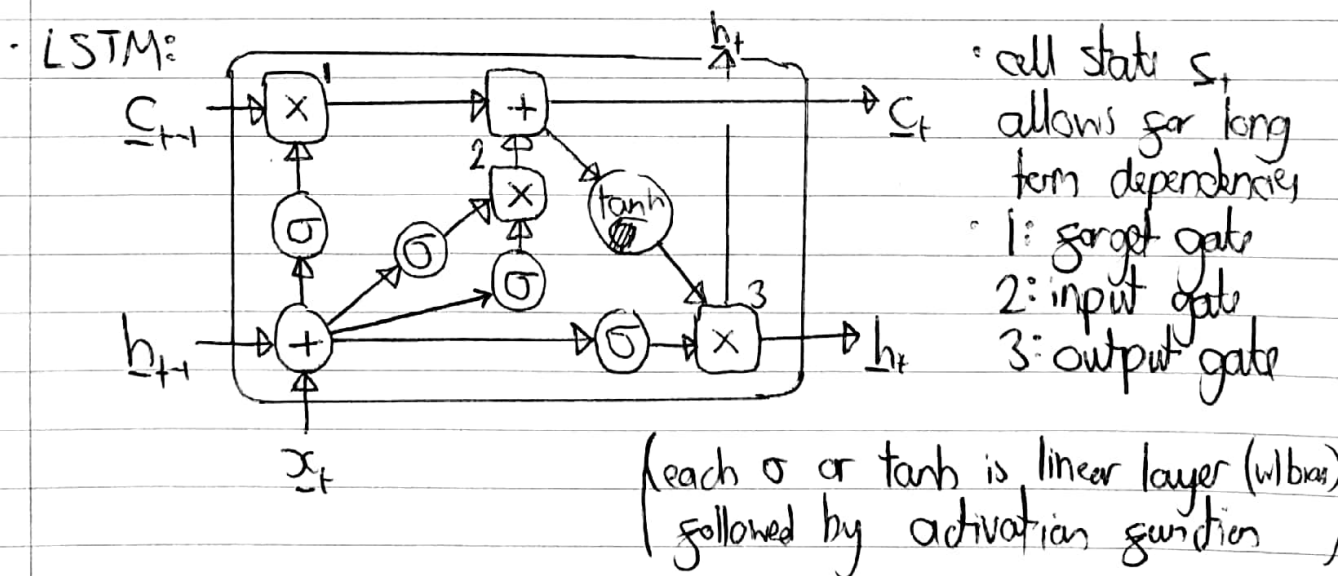
$$\frac{\partial L_t}{\partial \underline{B}} = \frac{\partial L_t}{\partial p_t} \frac{\partial p_t}{\partial h_t} \frac{\partial h_t}{\partial \underline{B}}$$

$$\begin{aligned}
 &= (p_t - \text{label}_t) h_t^T \Rightarrow \frac{\partial L}{\partial \underline{B}} = \sum_{t=1}^T \frac{\partial L_t}{\partial \underline{B}} \\
 \frac{\partial L_t}{\partial \underline{W}} &= \frac{\partial L_t}{\partial p_t} \frac{\partial p_t}{\partial h_t} \frac{\partial h_t}{\partial \underline{W}} = \sum_{k=1}^t \frac{\partial L_t}{\partial p_t} \frac{\partial p_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial \underline{W}}
 \end{aligned}$$

- problem: $h_t = W^T h_{t-1}$, $h_t \rightarrow \infty$ if $|W| > 1$
 $h_t \rightarrow 0$ if $|W| < 1$

(vanishing or exploding gradients)

makes learning long term dependencies very hard



- Can also help solve exploding gradients by thresholding:
 if $\|g\| > \text{threshold}$:
 $g \leftarrow \frac{\text{threshold}}{\|g\|} \times g$

Applications:

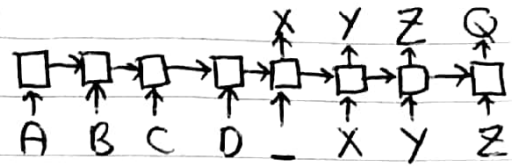
- Modelling languages; sequence of words

- Pixel prediction; image as sequence

- Sequence to sequence:

- image captioning, speech

- machine translation



Beyond Image Recognition: End-to-End Learning, Embeddings

• Pretraining:

- use a ready trained network, and use as base to build on
- don't update these params; just feed output into new network (or update slowly, low learning rate)
- eg. preusing network pre-trained on ImageNet for other image datasets

• Object detection:

- As classification; classify all boxes (may be repeated objects; slow)
- As prediction; regresses 4 corners of box; unknown no. boxes
- Bounding box proposals:
 - module predicts which boxes may be objects
 - then passes each box to a classifier

• Semantic Segmentation:

- label each pixel into an object class
- output full resolution map with 1 channel per class & probabilities for each pixel
 - 'fully convolutional networks'; no linear layers in ^{whole} network

• Spatial transformers:

- learn to transform input, eg rotate, to some canonical representation

• Similarity / dissimilarity

- inputs associated with each other; eg all images of a given persons face
- use different loss functions based on predicted similarity

• Maze navigation:

- problem: sparse rewards; we want some form of spatial knowledge;

- add auxillary rewards for auxillary tasks; depth prediction or loop closure prediction



Optimisation for Machine Learning

objective
↓
parameters

- Gradient Descent: $\underline{\Theta}_{k+1} = \underline{\Theta}_k - \alpha_k \nabla h(\underline{\Theta}_k)$
 • motivated by 1st order Taylor expansion
 $h(\underline{\Theta} + \underline{\delta}) \approx h(\underline{\Theta}) + \nabla h(\underline{\Theta})^T \underline{\delta}$
 • reasonable for small $\underline{\delta}$, ie low learning rates
 • problem: Oscillates along a valley
 • cannot solve by fine tuning learning rate; either big zig-zag or slow progress
 • reason: we minimize local quadratic approximation to $h(\underline{\Theta})$
 $h(\underline{\Theta} + \underline{\delta}) \approx h(\underline{\Theta}) + \nabla h(\underline{\Theta})^T \underline{\delta} + \frac{1}{2} \underline{\delta}^T H(\underline{\Theta}) \underline{\delta}$
 • approximate $H(\underline{\Theta})$ with $L\underline{I}$; treat all directions as having same curvature, $\frac{1}{2} \underline{\delta}^T (L\underline{I}) \underline{\delta} = \frac{L}{2} \|\underline{\delta}\|^2$
 $h(\underline{\Theta} + \underline{\delta}) \approx h(\underline{\Theta}) + \nabla h(\underline{\Theta})^T \underline{\delta} + \frac{L}{2} \|\underline{\delta}\|^2$
 $\Rightarrow -\frac{1}{L} \nabla h(\underline{\Theta}) = \underset{\underline{\delta}}{\operatorname{argmin}} (h(\underline{\Theta}) + \nabla h(\underline{\Theta})^T \underline{\delta} + \frac{L}{2} \|\underline{\delta}\|^2)$



asymptotic performance;
often stop before this point anyway
↓

Worst case:
Very pessimistic

Worst case
upper bound
→

- Convergence theory:
 $h(\underline{\Theta}_k) - h(\underline{\Theta}^*) \leq \frac{L}{2} \left(\frac{k-1}{k+1}\right)^{2k} \|\underline{\Theta}_0 - \underline{\Theta}^*\|^2$ (for $\alpha_k = \frac{2}{L+\mu}$)
 $\Rightarrow$ iterations for $h(\underline{\Theta}_k) - h(\underline{\Theta}^*) \leq \epsilon$: $k \in O(k \log \frac{1}{\epsilon})$ | $k = \frac{L}{\mu}$
- Momentum:
 $\underline{V}_{k+1} = \eta_k \underline{V}_k - \alpha_k \nabla h(\underline{\Theta}_k)$ η_k : friction constant
 $\underline{\Theta}_{k+1} = \underline{\Theta}_k + \underline{V}_{k+1}$ α_k : learning rate

• helps to fix zig-zagging problem by updating w/ moving average of gradient

• Convergence theory:

• another first order method (update is linear combination of previous gradient estimates)

• $k \in O(\sqrt{k} \log \frac{1}{\epsilon})$; $\sqrt{k}$ faster speedup
 worst case lower bound

← upper bound for Nesterov, but similar performance in practice

• Second order methods:

- Approximate $h(\underline{\theta} + \underline{\delta})$ w/ 2nd order Taylor expansion

$$h(\underline{\theta} + \underline{\delta}) \approx h(\underline{\theta}) + \nabla h(\underline{\theta})^T \underline{\delta} + \frac{1}{2} \underline{\delta}^T \underline{H} \underline{\delta}$$

$$\Rightarrow \underline{H}^{-1} \nabla h = \underset{\underline{\delta}}{\operatorname{argmin}} (h(\underline{\theta}) + \nabla h(\underline{\theta})^T \underline{\delta} + \frac{1}{2} \underline{\delta}^T \underline{H} \underline{\delta})$$

$$\underline{\theta}_{k+1} = \underline{\theta}_k - \underline{H}^{-1} \nabla h(\underline{\theta}_k)$$

• Problems:

- approximation only valid near $\underline{\theta}_k$
- 1st order methods approximate $\underline{H}$ w/ $L\underline{I}$, here we ~~approximate~~ use real $h(\underline{\theta})$, but this may underestimate curvature as we move away from $\underline{\theta}$
- Solution: constrain $\underline{\delta}$ to be in some region R , near $\underline{\theta}$ where our approximation remains good
 - if $R = \{\underline{\delta} : \|\underline{\delta}\|^2 \leq r\}$, then doing our $\underset{\underline{\delta} \in R}{\operatorname{argmin}}(\dots)$ is approximately equivalent to doing $\operatorname{argmin}(\dots)$, but replacing $\underline{H}(\underline{\theta})$ with $(\underline{H}(\underline{\theta}) + \lambda \underline{I})$, where λ is some function of r
- can also instead replace $\underline{H}$ with another matrix, upper bounding curvature in over larger regions
 - eg Generalized Gauss Newton Matrix
 - Fisher information matrix
 - Empirical Fisher info matrix

• Generalized Gauss Newton:

- Assume $h(\underline{\theta}) = \sum h_i(\underline{\theta}) = \sum \lambda(y_i, g(x_i, \underline{\theta}))$
- $\underline{G} = \sum \underline{J}_i^T \underline{H}_i \underline{J}_i$, $\underline{J}_i$ is Jacobian of $g(x_i, \underline{\theta})$ wrt $\underline{\theta}$
 $\underline{H}_i$ is Hessian of $\lambda(y_i, z_i)$ wrt z_i
- $\underline{G} = \underline{H}$ if replace $g(x_i, \underline{\theta})$ w/ 1st order approximations
- for square loss, $\underline{H}_i = \underline{I} \Rightarrow \underline{G} = \sum \underline{J}_i^T \underline{J}_i$
- $\underline{G}$ always PSD, more conservative than Hessian & works much better in practice

Optimisation 2

Curvature matrix approximations:

or
some other
curvature
matrix $\underline{B}$

- Θ can have 10s or millions of dimensions
- $\underline{H}$ is $n \times n$ & $O(n^3)$ to invert; impractical

Approximations:

- diagonal: $O(n)$ storage, $O(n)$ to invert

• only reasonable if eigenvalues of $\underline{B}$ closely aligned w/ coordinate axes

• popular is empirical Fisher (used by RMS-prop & Adam)

- low-rank: $O(rn)$ storage, computation & to invert

• diagonal + rank r correlations

• best if $\underline{B}$ has few important eigenvectors w/ large eigenvalues

• eg L-BFGS

- block-diagonal: $O(bn)$ to store, $O(b^2n)$ to invert

• can be awkward to compute (similar to diagonal)

• eg TONGA

- Kronecker-product: $O(n)$ to store/compute, $O(Bn)$ to invert

• block diagonal of GGN / Fisher

• blocks correspond to layers, each approximated by Kronecker-product of 2 smaller matrices

Stochastic optimisation:

- randomly sample mini-batch to compute gradient

• useful for momentum, but must be careful w/ α & η choice

• for second order methods, gives no asymptotic improvement
*←(and momentum)
• but pre-asymptotically can help, which is what we care most about

Attention & Memory in Deep Learning

- Deep networks have implicit attention; respond more to certain parts of input than to others (shown by Jacobian)

$$J_{ij} = \frac{\partial y_i}{\partial x_j} \quad (\text{compute w/ backprop; set loss to output activation})$$

- In RNNs, attention ^{shows} ~~gives~~ a ~~form of~~ memory; attention to past inputs to produce present output

• shown by sequential Jacobian; $\underline{J}_k^t = \left(\frac{\partial y_k^t}{\partial x_1}, \frac{\partial y_k^t}{\partial x_2}, \dots \right)$

- e.g. delayed dot of i in handwriting, or out-of-order machine translation

- Explicit attention:

- limit data presented to network in some way
- + ↑ computational efficiency, scalability, interpretability

Extra data

↓

Attention model

↓
Glimpse

Network

Output

Input

- attention model chooses what ~~it~~ should be visible to network in next time step; the 'glimpse'
- whole system recurrent, even if network isn't

Glimpse: $P(g|a)$

- generally define probability distribution over glimpses of data, given some set of attention outputs from the network

not differentiable

↓
hard attention; specific window passed to network

- can train this w/ RL techniques; reward = loss induced by the glimpse (can use RL whenever a module not differentiable)

- glimpse could e.g. be Gaussian w/ given mean & variance, over coordinates of an image, defining a window to pass to network

- Generally prefer soft attention; differentiable, can train end-to-end with backprop!

- take expectation from $P(g|a)$ rather than a sample

Deep Learning for NLP

• Word Vectors:

- sparse, orthogonal & semantically weak
- we want dense representations based on context/use of words
- Count-based:

- C = context words

- count words in C in window around a given word to determine its vector representation

- ~~need~~ need to normalise counts; distinguish informative high counts from uninformative (e.g. to -ids, $pmi...$)

- measure similarity by cosine similarity ($\cosine(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$)

• Neural embedding models:

$$\text{argmin}_{\theta, E} L(\theta, E), \quad L(\theta, E) = -\sum_t \sum_{t_i} \text{Score}(t_i, c(t_i), \theta, E)$$

E is embedding

t is a word in vocab

t_i is an instance of t in data

- Score function important;

(context)

- must embed t_i with E , be differentiable & take $c(t_i)$ into account

• Discrete Language Models:

- Goal: estimate $P(S_{1:n}) = \prod_{i=1}^n P(s_i | S_{1:i-1})$

- $S_{1:i-1}$ variable length; use n -gram (fixed window size)

- too small n = bad model, too large = counts too low; suffer from sparsity

• Continuous Condition Language Models:

- LBL/NLM:

- co-learn word embeddings, then use n -gram model on them

- log-bilinear model = linear from embeddings to probabilities

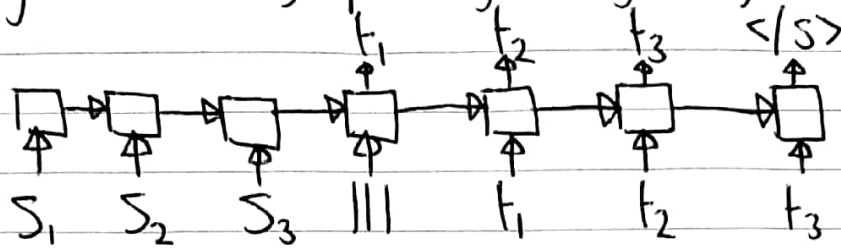
- neural language model = sigmoid activation function

- RNN-LMs:

- no longer need k -Markov assumption

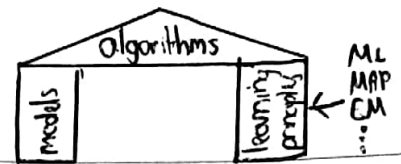
(in practice still some soft limitations on k based on data, memory size etc)

- Sequence-to-sequence mapping w/ RNNs:
 - eg translation, parsing (string $\rightarrow$ tree), or more general computation



- train to maximise likelihood of $\underline{t}$ given $\underline{S}$
- problems:
 - non-adaptive capacity, & modelling target sentence dominates training
 - encoder 'gradient starved' $\leftarrow$ (attention models can help here)
- Semantic Composition:
 - language tree structured; get parse tree distribution & exploit this with an RNN
 - ~~can, eg, annotate~~

Unsupervised Learning & Generative Models



• Probabilistic Machine Learning:

• Model types:

• Unconditional: $p(x)$, no targets / labels

• what generative model usually means

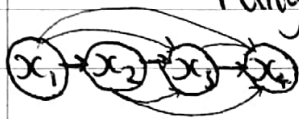
• Conditional $p(y|x)$

• supervised learning; regression & classification

all are actually generative models

• Types of generative models:

• Fully observed models:



• model data directly; don't introduce any unobserved variables

• directed graphical models:

+ simple to learn parameters (log-likelihood computable)

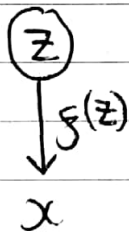
+ scales well

- order sensitive

• undirected hard to learn parameters

- slow to sample generate; need to go thru elements sequentially

• Implicit models:



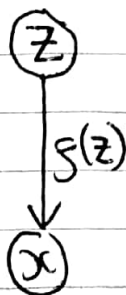
• transform unobserved noise source using parameterised function

+ easy to sample & compute expectations

- hard to maintain invertibility & to optimise

- lack of a noise model

• Latent variable models



• local random variables representing hidden causes

+ easy to sample

+ encode hierarchy, depth, structure

+ latent's compress & represent data

+ can score, compare & select models by likelihood

- inversion difficult

- marginal likelihood requires approximations

Inference in Prescribed Probabilistic Models:

Model evidence:

$$p(x) = \int p(x, z) dz$$

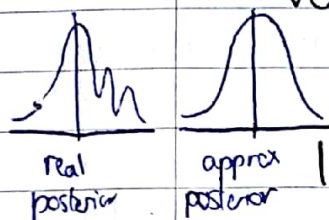
• aim to improve this for given data sets

• intractable in general; transform into expectation over known distribution

Jensen's inequality:

• for concave functions ξ ,
 $\xi(E(x)) \geq E(\xi(x))$

Variational Inference



Variational lower bound

$$p(x) = \int p(x|z)p(z)dz$$

$$p(x) = \int p(x|z) \frac{p(z)}{q(z)} q(z) dz$$

$$\log p(x) \geq \int q(z) \log(p(x|z) \frac{p(z)}{q(z)}) dz$$

$$= \int q(z) \log p(x|z) - \int q(z) \log \frac{p(z)}{q(z)} dz$$

$$= E_{q(z)} [\log p(x|z)] - KL[q(z) || p(z)]$$

Jensen's inequality

• $q(z)$ is our approximation to $p(z)$, using some known, simple distribution

• $KL(q||p)$ is a 'penalty' term, as $q \neq p$

• optimise for q ; eg is Gaussian, variational parameters are mean & variance to optimise for

• this can tighten the bound, getting closer to true marginal likelihood

max likelihood for others

• Only some parts of model given prob distributions, eg latent vars (in Variational Bayes, all unknown quantities are probability distributions)

+ applicable to all probabilistic models

+ convert integral problem to optimisation

+ Numerically stable & parallelisable, fast to converge

• need to choose form of q , and only approximate

- local minima

- can under estimate variance in posterior

Unsupervised Learning & Generative Models

• Family of Approximate Posterior distributions:

• Free form:

• get $q(z) \propto p(z) e^{\log p(x|z, \theta)}$

• problem: Solving is normalisation of original problem

• Fixed form:

• specify explicit form of q ; $q_\phi(z) = \mathcal{J}(z; \phi)$

• Mean field methods:

• assume distribution is factorised

• can introduce structure to factorisation (dependencies) to be more expressive

• eg autoregressive approximation: (condition on all previous variables)

$$q(z) = \prod q_i(z_i | z_{1:i-1})$$

• Variational Latent Gaussian models:

$$z \sim N(z | 0, 1), \quad y \sim p(y | f_\theta(z)), \quad q(z) = \prod N(z_i | \mu_i, \sigma_i^2)$$

$$F(y, q) = E_{q(z)} [\log p(y|z)] - KL[q(z) \| p(z)]$$

$$= E_{q(z)} [\log p(y|z)] - \sum_i KL[N(z_i | \mu_i, \sigma_i^2) \| N(z_i | 0, 1)]$$

$$= E_{q(z)} [\log p(y|z)] - \frac{1}{2} \sum_i [\sigma_i^2 + \mu_i^2 - 1 - \ln \sigma_i^2]$$

• Variational Optimisation:

• EM algorithm

• E step:

$$\phi \propto \nabla_\phi F(x, q)$$

(Variational params)

$$\phi_n \propto \nabla_\phi E_{q_{\phi}(z)} [\log p_\theta(x_n | z_n)] - \nabla_\phi KL[q(z_n) \| p(z_n)]$$

• M step:

$$\Theta \propto \nabla_\Theta F(x, q)$$

(model params)

$$\Theta \propto \frac{1}{N} \sum_n E_{q_{\phi}(z)} [\nabla_\Theta \log p_\Theta(x_n | z_n)]$$

• can do stochastic approach; do EM on minibatches

• more scalable

• If expected log likelihood & KL-divergence can't be calculated, use Monte-Carlo methods

• doubly stochastic estimation; stochasticity from minibatch & Monte-Carlo estimation

• Learning in Implicit Probabilistic Models:

• Estimation by comparison:

• compare estimate $q(x)$ with true $p^*(x)$

$$\frac{p^*(x)}{q(x)} = 1 \quad p^*(x) = q(x)$$

• test a hypothesis, then modify q slightly to better match p^* result

• Adversarial Learning:

Comparison loss $\rightarrow \Theta \propto \nabla_{\Theta} E_{p^*(x)} [\log D_{\Theta}(x)] + \nabla_{\Theta} E_{q_{\Phi}(x)} [\log (1 - D_{\Theta}(x))]$

Generative loss $\rightarrow \Phi \propto -\nabla_{\Phi} E_{q(z)} [\log (1 - D_{\Theta}(f_{\Phi}(z)))]$

$$\left(\frac{p^*(x)}{q(x)} = \frac{p(x|y=1)}{p(x|y=-1)} = \frac{p(y=1|x)}{p(y=-1|x)} = \frac{D_{\Theta}(x)}{1 - D_{\Theta}(x)} \right)$$